

CCAO-DSL: Six Levels of Complexity

Visual Reference for Claude Code Agent Orchestration DSL Date: 2025-10-19 ---

ASCII Complexity Pyramid

```
|| LEVEL 6: MATHEMATICAL META ||
|| Category Theory & Parameterized Workflows ||
||  $W(\tau)(f) = \lambda x. (f_1 \circ f_2 \circ \dots \circ f_n)(x)$  ||
|| Functors, Monads, Abstract Composition ||
```



```
|| LEVEL 5: WORKFLOW COMPOSITION & LOOPS ||
|| Named workflows calling workflows with parameters ||
|| workflow(params) -> sub_workflow -> map(fn) ||
|| Context sharing, dynamic selection, iteration ||
```



```
|| LEVEL 4: COMPLEX ORCHESTRATION & CONDITIONALS || | |
|| Nested parallel/sequential, if/else, error handling ||
|| research -> (design || implement) -> if(success) test : rollback ||
|| Resource constraints, retry logic, conditional branches ||
```



```
|| LEVEL 3: PARALLEL STREAMS + SEQUENTIAL (YOUR EXAMPLE) || | | | |
|| Multiple parallel execution streams with task specification ||
|| (/deep + /ctx7 + /research || /orch /wflw /coord || /meta-skill) : task ||
|| 3 parallel streams, skill combination, complex coordination ||
```



```
|| LEVEL 2: SIMPLE COMBINATION & BASIC OPERATORS || | |
|| Single operator usage: combination (+), sequence (->), parallel (||) ||
|| agent1 + skill1 (combination) ||
|| agent1 -> agent2 (sequence) ||
|| agent1 || agent2 (parallel) ||
```



```
LEVEL 1: BASIC SINGLE INVOCATION
Single agent, skill, or command with optional parameters
api-architect : "design REST API"
/ctx7 = express
```

Detailed Level Breakdown

Level 1: Basic Single Invocation

Complexity: Minimal **Execution Model:** Single function call **Token Budget:** 5,000-15,000

Time Estimate: 2-5 minutes ##### Syntax Pattern

```
<agent> : <task>
<command> = <parameter>
<skill>
```

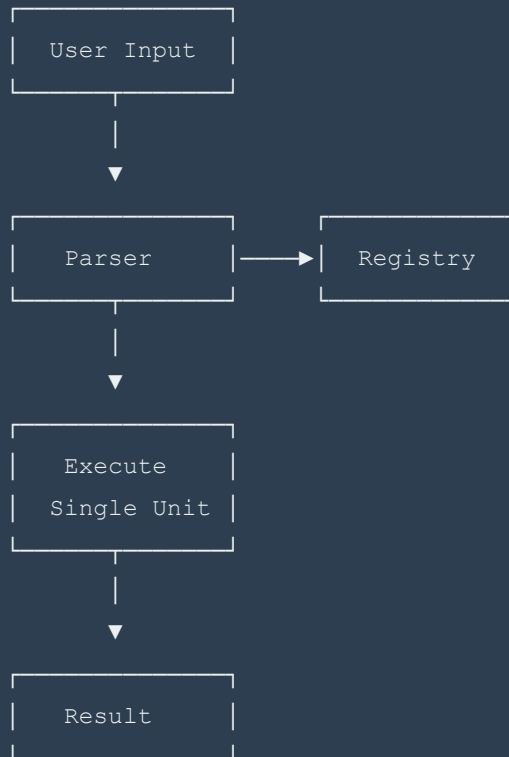
Examples

```
// Single agent invocation
api-architect : "design REST API for user management"

// Single command execution
/ctx7 = fastapi

// Single skill activation
postgresql-database-engineering
```

Execution Flow



Mathematical Model

```
f: Input → Output  
Simple function application with no composition
```

Level 2: Simple Combination & Basic Operators

Complexity: Low **Execution Model:** Binary operations **Token Budget:** 10,000-30,000 **Time Estimate:** 5-15 minutes #### Syntax Pattern

```
<agent> + <skill>           // Combination  
<agent> -> <agent>          // Sequence  
<agent> || <agent>          // Parallel
```

Examples

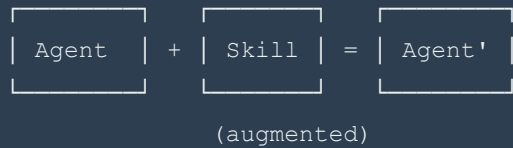
```
// Combination: Agent with skill augmentation
api-architect + rest-api-design-patterns + postgresql

// Sequence: Pipeline of two agents
deep-researcher -> api-architect

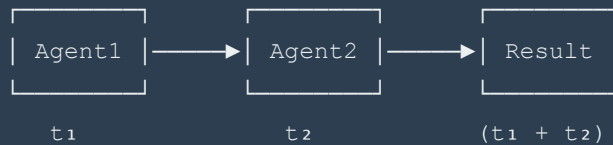
// Parallel: Two agents running concurrently
frontend-architect || backend-architect
```

Execution Flow

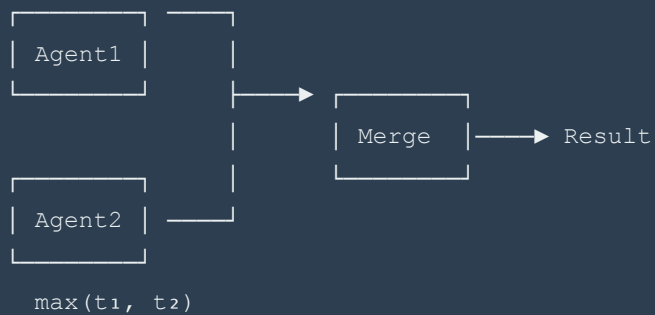
COMBINATION (+):



SEQUENCE (->):



PARALLEL (||):



Mathematical Model

```
Combination:   $S_1 \oplus S_2 \rightarrow S_3$ 
Sequence:      $f \circ g = \lambda x. f(g(x))$ 
Parallel:      $f \otimes g = \lambda x. (f(x), g(x))$ 
```

Operator Properties

COMBINATION (+):

- Associative: $(A + B) + C = A + (B + C)$
- Commutative: $A + B = B + A$
- Identity: $A + \emptyset = A$

SEQUENCE (->):

- Associative: $(A \rightarrow B) \rightarrow C = A \rightarrow (B \rightarrow C)$
- NOT Commutative: $A \rightarrow B \neq B \rightarrow A$
- Identity: $\text{id} \rightarrow A = A$

PARALLEL (||):

- Associative: $(A || B) || C = A || (B || C)$
- Commutative: $A || B = B || A$

Level 3: Parallel Streams + Sequential ⭐ YOUR EXAMPLE

Complexity: Medium **Execution Model:** Multi-stream DAG execution **Token Budget:** 40,000-100,000 **Time Estimate:** 20-45 minutes ##### Syntax Pattern

```
(<stream1> || <stream2> || <stream3>) : <task>
```

Where each stream can be:

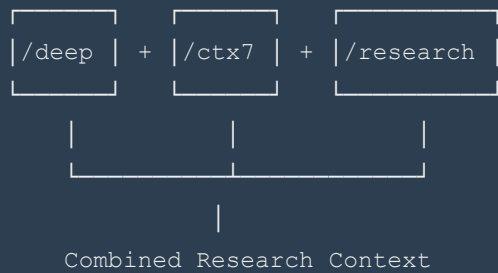
- Single agent: agent
- Combination: agent + skill
- Sequence: cmd -> cmd -> cmd

Your Example Deconstructed

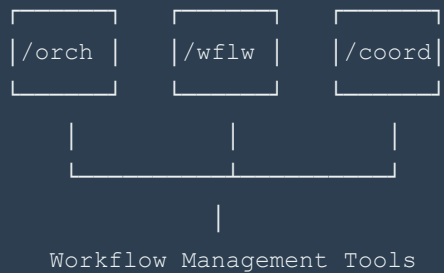
```
(/deep + /ctx7 + /research || /orch /wflw /coord || /meta-skill-builder || /meta-agent)  
: DSL for Claude code
```

Parallel Stream Analysis

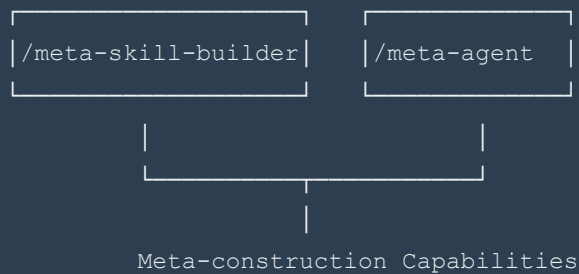
STREAM 1: Deep Research Foundation



STREAM 2: Orchestration Tools

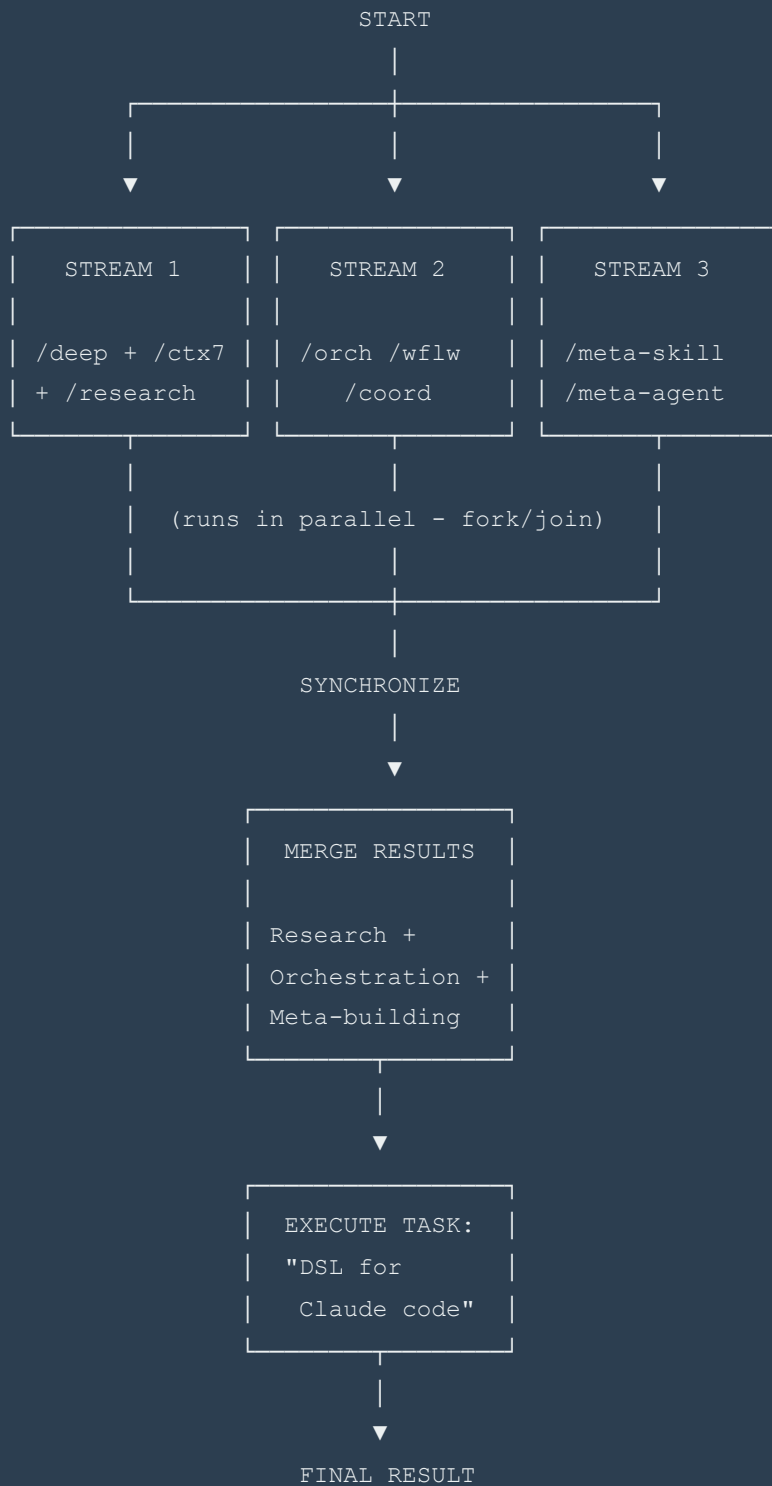


STREAM 3: Meta-builders

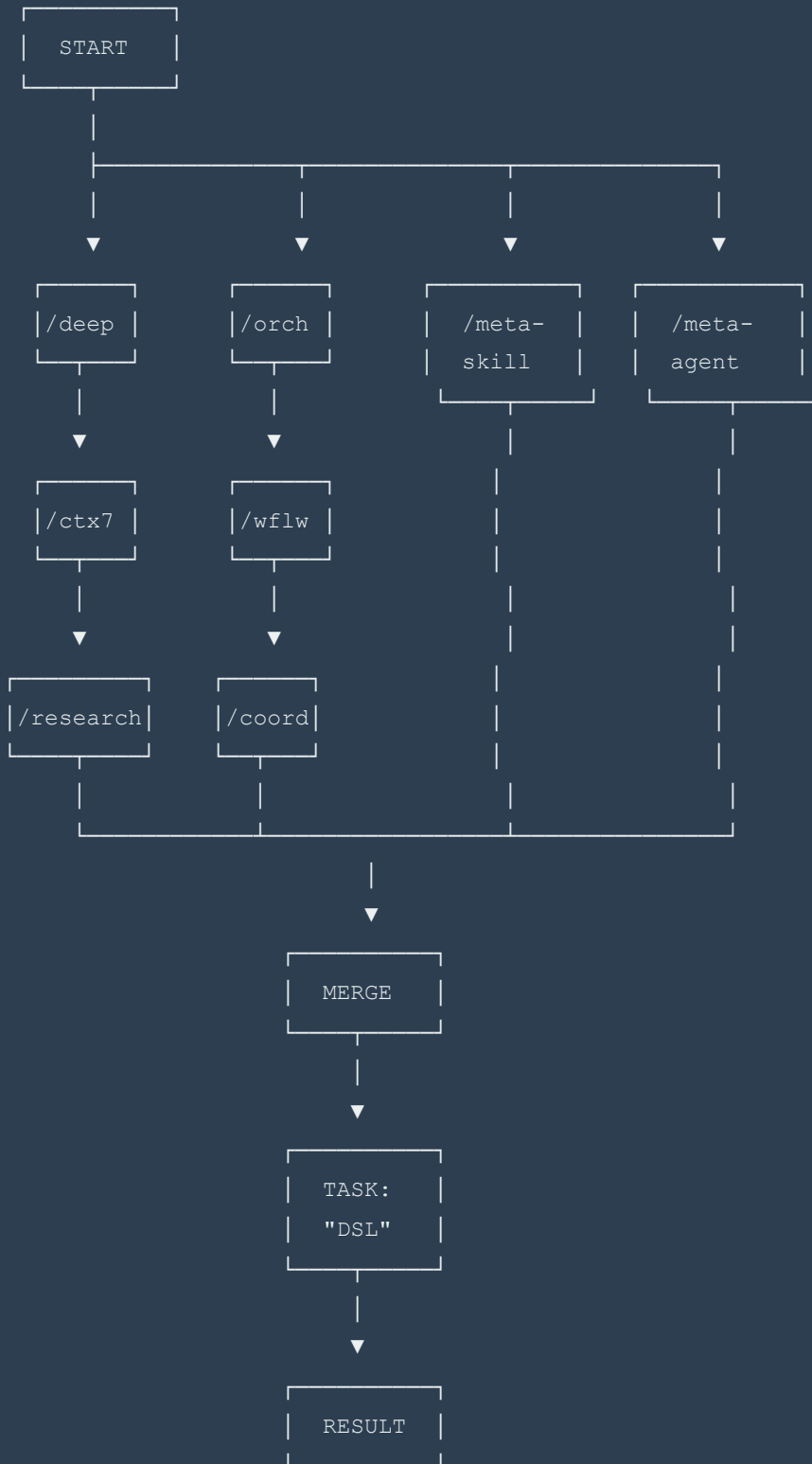


TASK SPECIFICATION:

Execution Flow



DAG Representation



Mathematical Model

Let:

$S_1 = \text{/deep} + \text{/ctx7} + \text{/research}$

$S_2 = \text{/orch /wflw /coord}$

$S_3 = \text{/meta-skill-builder}$

$S_4 = \text{/meta-agent}$

Execution:

$W = (S_1 \parallel S_2 \parallel S_3 \parallel S_4) \circ \text{task}$

Where:

$(S_1 \parallel S_2 \parallel S_3 \parallel S_4) = \text{fork}([S_1, S_2, S_3, S_4]) \rightarrow \text{join} \rightarrow \text{merge}$

Time complexity:

$T(W) = \max(T(S_1), T(S_2), T(S_3), T(S_4)) + T(\text{merge}) + T(\text{task})$

Resource Allocation

```

stream_1:
  agents: [deep-researcher, context7-doc-reviewer]
  commands: [/deep, /ctx7, /research]
  estimated_tokens: 25,000
  estimated_time: 15-20 min

stream_2:
  commands: [/orch, /wflw, /coord]
  estimated_tokens: 15,000
  estimated_time: 10-15 min

stream_3:
  commands: [/meta-skill-builder, /agent]
  estimated_tokens: 20,000
  estimated_time: 12-18 min

total_parallel:
  max_tokens: 60,000 (not summed, concurrent)
  max_time: ~20 minutes (longest stream)

task_execution:
  tokens: 30,000
  time: 15-25 min

grand_total:
  tokens: ~90,000
  time: ~45 minutes

```

Level 4: Complex Orchestration & Conditionals

Complexity: High **Execution Model:** DAG with branching logic **Token Budget:** 80,000-150,000 **Time Estimate:** 45-90 minutes ##### Syntax Pattern

```
<agent> -> (<parallel_group>) -> if(<condition>) <then_branch> : <else_branch>
```

Examples

```
// Nested parallel within sequence
research -> (design || implement || test) -> integrate

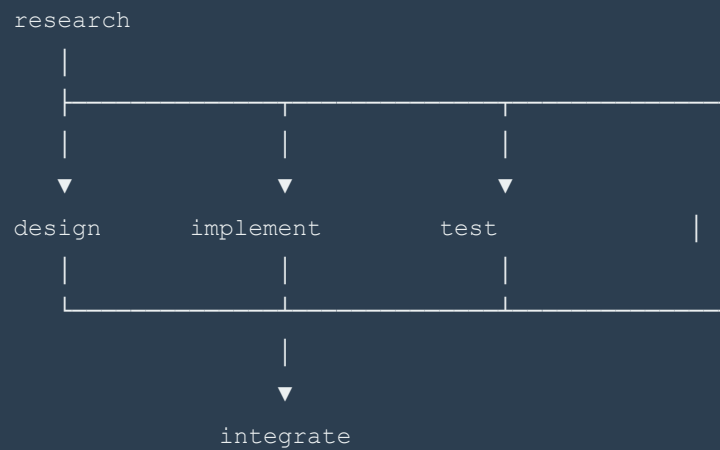
// Conditional execution
deploy -> if(tests_pass) production : rollback

// Error handling with retry
api_call -> catch(error) retry(3) : fail_gracefully

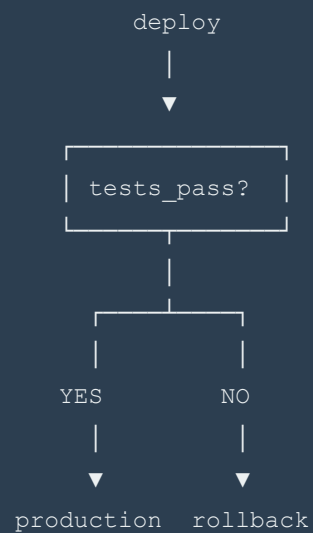
// Resource-constrained execution
(heavy_task1 || heavy_task2)[budget=50000]
```

Execution Flow

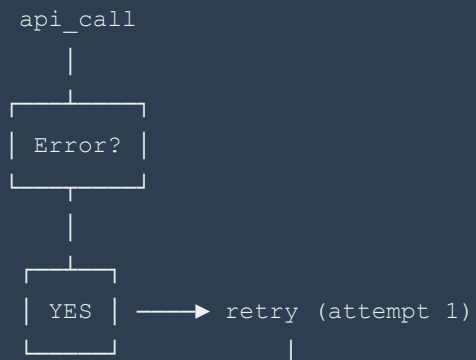
NESTED PARALLEL IN SEQUENCE:



CONDITIONAL BRANCHING:



ERROR HANDLING:





Advanced Features 1. Conditional Execution

```
workflow test_and_deploy {
  test_result = run_tests

  if (test_result.success) {
    deploy -> monitor
  } else {
    notify_team -> fix_bugs -> retry
  }
}
```

2. Loop Constructs

```
workflow process_all_files {
  files = glob("*.py")

  for file in files {
    analyze(file) || test(file)
  }

  aggregate_results
}
```

3. Resource Constraints

```
// Limit token budget for parallel execution
(agent1 || agent2 || agent3)[
  budget = 50000,
  timeout = 30min,
  retry = 2
]
```

Mathematical Model

```

Conditional:
  if(p) a else b = p ? a : b

  where p: Context → Boolean
        a, b: Workflow

Loop:
  for x in xs: f(x) = map(f, xs)

  where xs: List(τ)
        f: τ → Result

Resource Constraint:
  constrained(w, r) = w with resource_limit(r)

  where w: Workflow
        r: {budget, timeout, ...}

```

Level 5: Workflow Composition & Iteration

Complexity: Very High **Execution Model:** Higher-order workflows **Token Budget:** 120,000-250,000 **Time Estimate:** 90-180 minutes ##### Syntax Pattern

```

workflow <name>(<params>) {
  <sub_workflow1>(<args>) ->
  <sub_workflow2>(<args>) ->
  map(<collection>, <workflow_fn>)
}

```

Examples

```

// Named workflow with parameters
workflow full_stack_app(spec, tech_stack) {
  research(spec) ->
  design_api(tech_stack.backend) ->
  design_ui(tech_stack.frontend) ->
  (implement_backend || implement_frontend) ->
  integrate ->
  test ->
  deploy
}

// Workflow calling workflow
workflow ci_cd_pipeline {
  build_workflow() ->
  test_workflow() ->
  if(tests_pass) {
    deploy_workflow(environment="staging") ->
    smoke_test_workflow() ->
    if(smoke_pass) {
      deploy_workflow(environment="production")
    }
  }
}

// Map-reduce pattern
workflow analyze_repository {
  files = glob("src/**/*.py")

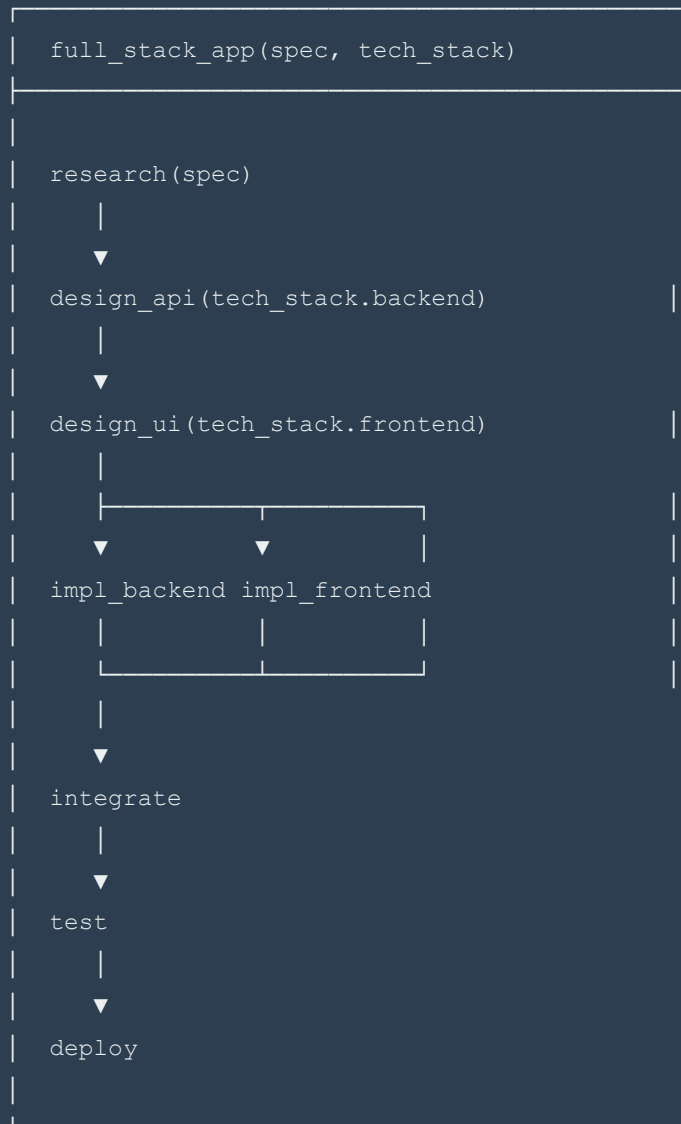
  // Map: analyze each file in parallel
  analyses = map(files, file => {
    code_quality(file) ||
    security_scan(file) ||
    test_coverage(file)
  })

  // Reduce: aggregate results
  report = reduce(analyses, merge_reports)
}

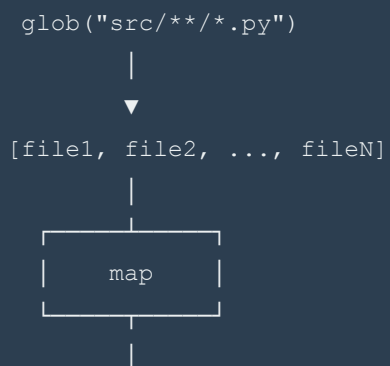
```

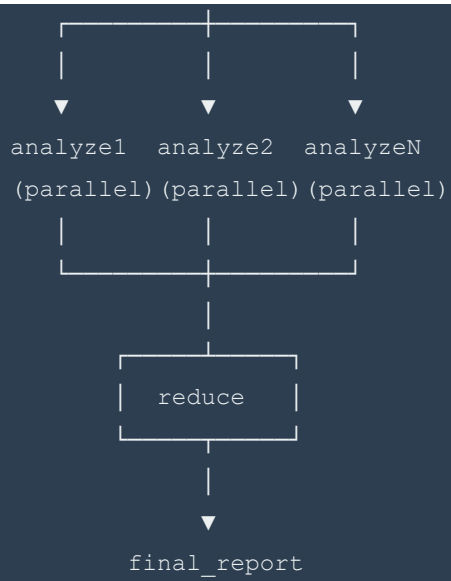
Execution Flow

WORKFLOW COMPOSITION:



MAP-REDUCE:





Context Sharing Pattern

```
workflow api_with_context {  
  // Build shared context  
  ctx = {}  
  ctx = ctx + /ctx7("fastapi")  
  ctx = ctx + /ctx7("postgresql")  
  ctx = ctx + /ctx7("redis")  
  
  // All agents share context  
  researcher[ctx] ->  
  architect[ctx] ->  
  implementer[ctx] ->  
  tester[ctx]  
  
  // Context contains all library docs  
}
```

Mathematical Model

Named Workflow:

$$W\langle \tau_1, \dots, \tau_n \rangle : (\tau_1 \times \dots \times \tau_n) \rightarrow \text{Result}$$

Workflow Composition:

$$W_1 \circ W_2 = \lambda x. W_1(W_2(x))$$

Map:

$$\text{map}: \text{List}(\tau) \times (\tau \rightarrow \sigma) \rightarrow \text{List}(\sigma)$$
$$\text{map}(xs, f) = [f(x_1), f(x_2), \dots, f(x_n)]$$

Reduce:

$$\text{reduce}: \text{List}(\tau) \times (\tau \times \tau \rightarrow \tau) \rightarrow \tau$$
$$\text{reduce}([x_1, \dots, x_n], f) = f(\dots f(f(x_1, x_2), x_3) \dots, x_n)$$

Context Sharing:

$$W[\text{ctx}]: (\text{Context} \times \text{Input}) \rightarrow (\text{Context}' \times \text{Output})$$

Level 6: Mathematical Meta-Programming

Complexity: Extreme **Execution Model:** Abstract workflow generators **Token Budget:** 200,000+ **Time Estimate:** 3+ hours ##### Syntax Pattern

```
meta_workflow <name>(\tau) = \lambda(params) -> workflow_ast

functor map_workflow: Workflow(\tau) \to Workflow(\sigma)

monad compose_workflows: List(Workflow) \to Workflow
```

Examples

```

// Parameterized workflow generator
meta_workflow microservice{Domain} {
  // Abstract over domain type
  type_check Domain

  // Generate workflow based on domain
  λ(domain: Domain) -> {
    design_schema(domain) ->
    generate_api(domain) ->
    implement_crud(domain) ->
    generate_tests(domain) ->
    deploy_service(domain)
  }
}

// Instantiate concrete workflows
user_service = microservice{User}("user management")
product_service = microservice{Product}("product catalog")

// Functor: map over workflow results
workflow_functor = map_workflow(
  base_workflow,
  transform_fn: (result) -> enhanced_result
)

// Monad: compose workflows with context threading
composed = do {
  r1 <- research_workflow
  r2 <- design_workflow(r1)
  r3 <- implement_workflow(r2)
  return r3
}

```

Category Theory Foundations

```

// Workflows form a category
Category Workflow where
  objects = {Agent, Skill, Result}
  morphisms = {Workflow{ $\tau_1$ ,  $\tau_2$ }}

  // Identity morphism
  id:  $\tau \rightarrow \tau$ 
  id(x) = x

  // Composition
  ( $\circ$ ): ( $\beta \rightarrow \gamma$ )  $\rightarrow$  ( $\alpha \rightarrow \beta$ )  $\rightarrow$  ( $\alpha \rightarrow \gamma$ )
  (f  $\circ$  g)(x) = f(g(x))

  // Laws:
  // 1. Left identity: id  $\circ$  f = f
  // 2. Right identity: f  $\circ$  id = f
  // 3. Associativity: (f  $\circ$  g)  $\circ$  h = f  $\circ$  (g  $\circ$  h)

// Functors between categories
Functor F: Workflow  $\rightarrow$  EnhancedWorkflow where
  fmap: ( $\alpha \rightarrow \beta$ )  $\rightarrow$  (F  $\alpha \rightarrow$  F  $\beta$ )

// Applicative Functor
Applicative F where
  pure:  $\alpha \rightarrow$  F  $\alpha$ 
  (<*>): F ( $\alpha \rightarrow \beta$ )  $\rightarrow$  F  $\alpha \rightarrow$  F  $\beta$ 

// Monad
Monad M where
  return:  $\alpha \rightarrow$  M  $\alpha$ 
  (>>=): M  $\alpha \rightarrow$  ( $\alpha \rightarrow$  M  $\beta$ )  $\rightarrow$  M  $\beta$ 

  // Laws:
  // 1. Left identity: return a >>= f = f a
  // 2. Right identity: m >>= return = m
  // 3. Associativity: (m >>= f) >>= g = m >>= ( $\lambda x \rightarrow$  f x >>= g)

```

Advanced Meta-Programming

```

// Dynamic workflow construction
meta build_workflow(capabilities: List{Capability}) -> Workflow {
  // Select agents based on capabilities
  agents = select_agents(capabilities)

  // Optimize execution order
  dag = build_dependency_graph(agents)
  schedule = optimize_schedule(dag)

  // Generate workflow AST
  return workflow_from_schedule(schedule)
}

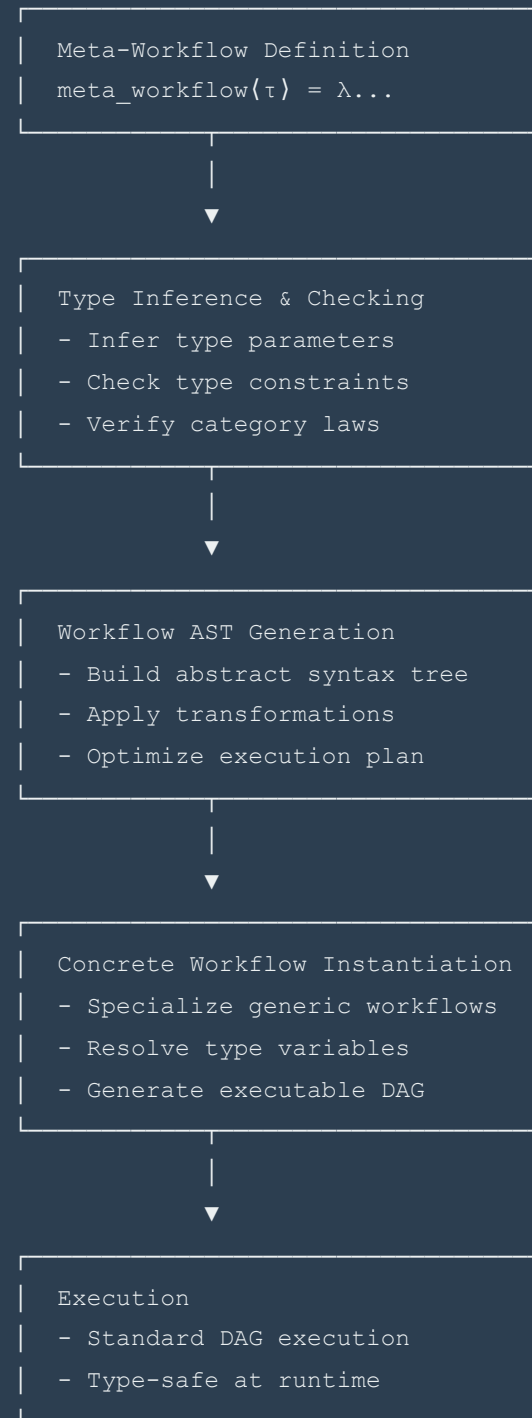
// Type-level computation
type family OptimalAgent{τ} where
  OptimalAgent{API} = api-architect
  OptimalAgent{Frontend} = frontend-specialist
  OptimalAgent{Backend} = backend-specialist

// Dependent types
workflow type_safe_pipeline{τ1, τ2, τ3}(
  input: τ1,
  f: τ1 → τ2,
  g: τ2 → τ3
) -> τ3 {
  result = g(f(input))
  return result : τ3 // Type guaranteed at compile time
}

```

Execution Model

META-WORKFLOW COMPILATION:



Mathematical Formalization

```
Abstract Workflow Type:
  W(τ₁, ..., τₙ) : (Context × τ₁ × ... × τₙ) → Result(σ)

Higher-Kinded Types:
  F(_) : Type → Type
  Example: Workflow(_), Result(_), List(_)

Type Constructor Composition:
  (F ∘ G)(τ) = F(G(τ))
  Example: Result(List(Workflow(Agent)))

Polymorphic Workflows:
  ∀τ. W(τ) : τ → Result(τ)
  Example: identity workflow works for any type

Dependent Types:
  W : Π(x : τ₁). τ₂(x)
  Example: pipeline length determines result type
```

Complexity Comparison Table

Level	Name	Operators	Control Flow	Composition	Token Budget	Time Est.	Example
1	Basic	None					
2	Simple	+, -, \ \					
3	Parallel Streams	+, -, \ \ , ()					
4	Multi-stream	N-ary					
5	Workflows	All + if, catch	Conditional	Nested			
6	Meta	All + {τ}, λ					

Progression Path

LEARNING PROGRESSION:

Level 1: Basic

↳ Master single invocations

↳ Level 2: Simple Operators

↳ Combine agents with skills

↳ Level 3: Parallel Streams ★

↳ Orchestrate multiple streams

↳ Level 4: Complex Orchestration

↳ Add conditionals and error handling

↳ Level 5: Workflow Composition

↳ Build reusable workflows

↳ Level 6: Meta-Programming

↳ Generate workflows programmatically

TYPICAL USER JOURNEY:

Week 1: Levels 1-2 (basics)

Week 2-3: Level 3 (parallel coordination) ★

Week 4-6: Level 4 (complex orchestration)

Month 2+: Level 5 (workflow library)

Month 3+: Level 6 (meta-workflows, if needed)

Visual Summary



Recommendations by Use Case

Quick Task (Use Level 1-2)

```
/ctx7 = express
api-architect : "design auth API"
```

Research & Planning (Use Level 3) ★

```
(deep-researcher || context7-doc-reviewer || api-architect) : "research payment systems"
```

Full Feature Development (Use Level 4-5)

```
workflow feature {  
  research -> (design || test-plan) -> implement -> test -> deploy  
}
```

Reusable Workflow Library (Use Level 5)

```
workflow microservice(domain) {  
  design_api(domain) ->  
  implement(domain) ->  
  test(domain) ->  
  deploy(domain)  
}
```

Advanced Automation (Use Level 6)

```
meta_workflow<Domain> = λd. optimize(generate_workflow(d))
```

--- **Your complexity level (Level 3) is optimal for most real-world orchestration tasks!**